TAREA 2: TABLA DE COMPLEJIDADES

Número	Nombre	Estructura principal usada	Complejidad Temporal	Complejidad espacial
0232	Implement Queue Using Stacks	Se hizo uso de dos colas	O(n)	O(n)
0239	Sliding Window Maximum	Se hizo uso de una cola de doble salida así como una lista	O(n)	O(n)
0622	Design Circular Queue	Se hizo uso una cola así como índices y contadores	O(1)	O(n)
0929	Unique Email Addresses	re.sub	O(n)	O(n)
0592	Fraction Addition and Substraction	re.findall; re.split	O(n)	O(n)
0065	Valid Number	re.match	O(n)	O(1)
0020	Valid Parentheses	Uso de una lista como pila	O(n)	O(n)
0150	Evaluate Reverse Polish Notions	Uso de una lista como pila y operaciones de extracción	O(n)	O(n)
0739	Daily Temperatures	Uso de una pila monótona de índices	O(n)	O(n)
0001	Two Sum	Array (lista)	O(n)	O(n)
0238	Product of Array Except Self	Lista (arreglo)	O(n)	O(1)
0056	Merge Intervals	Listas de listas (arreglo de intervalos)	O(n logn)	O(n)

Análisis:

¿Por qué list.pop(0) es O(n) y deque.popleft() es O(1)?

list.pop(0) es O(n) porque los elementos de una lista están almacenados en posiciones contiguas de memoria. Al eliminar el primer elemento, Python tiene que desplazar todos los elementos restantes a una posición a la izquierda para cubrir el espacio vacío. Por otro lado, deque.popleft() es O(1) porque utiliza una estructura basada en bloques enlazados, donde basta con desvincular el primer nodo y actualizar el puntero inicial sin necesidad de mover ningún otro elemento.

¿En qué problema les costó más trabajo elegir la estructura correcta y por qué.

La principal problemática que se encontró fue la asignación de variables correctas para cada clase, así como en un inicio empezar a darle visualización al código, pues se tenía la idea pero el inicio de la implementación fue algo complicado, posteriormente la siguiente complicación fue encadenar y relacionar de manera correcta todas las variables, asimismo existían formas “sencillas” de soluciones posibles, sin embargo al establecer las estructuras como colas, pilas y listas las soluciones se complican, pues se debe cumplir con las condiciones de estas estructuras, además de en todo momento cuidar las complejidades, tanto temporales como espaciales.

¿Qué ventaja tiene re.fullmatch sobre re.match para validar formatos? Da un ejemplo en el que den resultados distintos.

re.fullmatch() verifica que toda la cadena cumpla el patrón, mientras que re.match() solo verifica desde el inicio.
Por ejemplo, con el patrón \d+ y la cadena "123abc", re.match() encuentra una coincidencia con "123", pero re.fullmatch() no acepta la cadena completa. Por eso, re.fullmatch() es más útil para validar formatos.